

Rapport de Stage

Plateforme de Gestion de Bugs et de Feedback

Stagiaire : Yassir

Annee universitaire : 2025 / 2026

Technologies : Symfony 7.4 | PHP 8.2 | Doctrine ORM | SQLite | Twig | Hotwired Turbo

Table des Matieres

Introduction generale

Chapitre 1 : Présentation du stage et de l'entreprise

- 1.1 Contexte du stage
- 1.2 Presentation de l'entreprise d'accueil
- 1.3 Mission confiee

Chapitre 2 : Presentation du projet

- 2.1 Objectifs du projet
- 2.2 Cahier des charges fonctionnel
- 2.3 Roles et utilisateurs

Chapitre 3 : Architecture technique

- 3.1 Stack technologique
- 3.2 Structure du projet
- 3.3 Patron de conception MVC

Chapitre 4 : Modele de donnees

- 4.1 Diagramme Entite-Relation
- 4.2 Description des entites
- 4.3 Enums et workflow des statuts

Chapitre 5 : Fonctionnalites detaillees

- 5.1 Authentification et gestion des sessions
- 5.2 Tableaux de bord par role
- 5.3 Gestion des rapports de bugs
- 5.4 Systeme de filtrage
- 5.5 Gestion des projets et utilisateurs
- 5.6 Systeme de commentaires
- 5.7 Upload de captures d'ecran

Chapitre 6 : Securite

- 6.1 Authentification et hachage des mots de passe
- 6.2 Controle d'accès basé sur les rôles
- 6.3 Protection CSRF
- 6.4 Validation des entrées

Chapitre 7 : Tests

- 7.1 Tests unitaires
- 7.2 Tests fonctionnels

Conclusion generale

Annexes

Introduction generale

Le presente rapport detaille le stage realise dans le cadre de ma formation en developpement web. Ce stage avait pour objectif de mettre en pratique les connaissances theoriques acquises au cours de ma formation, tout en decouvrant les realites du developpement logiciel en entreprise.

La mission qui m'a ete confiee consistait a concevoir et developper une plateforme web de gestion de bugs et de feedback. Cette application interne permet aux equipes de development, aux clients et aux administrateurs de collaborer efficacement dans le suivi et la resolution des anomalies logicielles.

Ce rapport est organise en sept chapitres. Le premier presente le contexte du stage et l'entreprise d'accueil. Le deuxieme decrit le projet et ses objectifs. Le troisieme detaille l'architecture technique retenue. Le quatrieme expose le modele de donnees. Le cinquieme decrit les fonctionnalites implementees. Le sixieme aborde les aspects de securite. Enfin, le septieme presente la strategie de tests.

Chapitre 1 : Presentation du stage et de l'entreprise

1.1 Contexte du stage

Ce stage s'inscrit dans le cadre de la fin de formation en developpement web. D'une duree de plusieurs semaines, il m'a permis de travailler sur un projet concret de A a Z, depuis l'analyse des besoins jusqu'au deploiement de l'application.

L'objectif principal etait de developper une application web complete en utilisant des technologies modernes, tout en appliquant les bonnes pratiques de developpement logiciel : architecture MVC, tests, securite, et gestion de version avec Git.

1.2 Presentation de l'entreprise d'accueil

L'entreprise d'accueil est une organisation specialisee dans le developpement logiciel. Elle intervient dans differents domaines incluant le web, le mobile et les API. L'equipe utilise des methodologies agiles et des outils modernes pour garantir la qualite de ses livrables.

L'entreprise possede plusieurs projets actifs : un site web marketing, une application mobile de commande, un tableau de bord CRM interne, et une API REST pour les partenaires externes.

1.3 Mission confiee

Ma mission consistait a concevoir et developper une plateforme centralisee de gestion de bugs et de feedback. Cette application devait permettre :

- La creation et le suivi des rapports de bugs par les clients/testeurs
- L'assignation et le traitement des bugs par les developpeurs
- La supervision et l'administration globale par les administrateurs
- La gestion des projets et des utilisateurs
- Le systeme de commentaires pour faciliter la communication

Chapitre 2 : Presentation du projet

2.1 Objectifs du projet

La plateforme de gestion de bugs vise a resoudre les problemes lies a la communication des anomalies dans un contexte de developpement logiciel multi-acteurs. Les objectifs principaux sont :

- Centraliser tous les rapports de bugs dans un seul outil
- Automatiser le workflow de resolution des bugs
- Fournir des tableaux de bord pour chaque type d'utilisateur
- Assurer la traibilite complete d'un bug (creation a resolution)
- Faciliter la collaboration via les commentaires et les captures d'ecran

2.2 Cahier des charges fonctionnel

Les fonctionnalites principales de l'application sont :

- Authentification securisee par email et mot de passe
- Tableaux de bord personnalises par role (Admin, Developpeur, Client)
- CRUD complet pour les rapports de bugs avec champs detailles
- Systeme de filtrage avance (mot-cle, projet, statut, priorite, developpeur, dates)
- Gestion des statuts avec workflow defini (Open -> InProgress -> Fixed/Rejected)
- Assignment des bugs aux developpeurs par l'administrateur
- Systeme de commentaires par thread de bug
- Upload et affichage de captures d'ecran
- Gestion complete des projets et des utilisateurs (admin)
- Statistiques et metriques sur les tableaux de bord

2.3 Roles et utilisateurs

L'application gere trois roles distincts avec des niveaux d'accès différents :

ROLE_ADMIN (Administrateur)

- Accès complet a toutes les fonctionnalites
- Gestion des projets (creation, modification, suppression)
- Gestion des utilisateurs (creation, modification, desactivation)
- Assignment des bugs aux developpeurs
- Modification de la priorite et du statut de n'importe quel bug
- Suppression des bugs
- Tableau de bord avec statistiques globales

ROLE_DEVELOPEUR (Developpeur)

- Visualisation de tous les rapports de bugs
- Mise a jour du statut des bugs qui lui sont assignes
- Ajout de commentaires sur les bugs
- Tableau de bord avec compteur des bugs assignes

ROLE_CLIENT (Client / Testeur)

- Creation de nouveaux rapports de bugs
- Visualisation de ses propres rapports de bugs
- Ajout de commentaires sur ses bugs
- Tableau de bord avec compteur des bugs signales

Chapitre 3 : Architecture technique

3.1 Stack technologique

La plateforme a été développée avec les technologies suivantes :

Composant	Technologie	Version
Langage	PHP	>= 8.2
Framework	Symfony	7.4
ORM	Doctrine ORM	3.6
Moteur de templates	Twig	3.x
Base de données	SQLite	Dev / PostgreSQL (prod)
Frontend	AssetMapper + Turbo	8.0
Authentification	Symfony Security	7.4
Validation	Symfony Validator	7.4
Tests	PHPUnit	13.x

3.2 Structure du projet

La structure des répertoires suit les conventions Symfony :

Répertoire	Description
src/Entity/	Entités Doctrine ORM (User, BugReport, BugComment, Project)
src/Enum/	Enumerations PHP 8.1+ (BugStatus, BugPriority)
src/Controller/	Contrôleurs Symfony (routes et logique métier)
src/Form/	Types de formulaire Symfony
src/Repository/	Repositories Doctrine (requêtes DQL)
src/Service/	Services métier (FileUploader)
src/Security/	Composants de sécurité (UserChecker)
templates/	Templates Twig (vues HTML)
config/packages/	Configuration des bundles Symfony
migrations/	Migrations Doctrine ORM
tests/	Tests PHPUnit
assets/	Assets JavaScript (Stimulus, Turbo)

3.3 Patron de conception MVC

L'application suit le patron Model-View-Controller (MVC) propre à Symfony. Le modèle est représenté par les entités

Doctrine ORM, les vues sont des templates Twig, et les controleurs gerent la logique applicative et les routes.

Le flux de traitement suit le cycle classique : une requete HTTP arrive, le routeur Symfony l'associe a une methode de controleur, le controleur interagit avec le modele (entites, repositories, services), et retourne une reponse constituee d'un template Twig rendu.

Chapitre 4 : Modele de donnees

4.1 Diagramme Entite-Relation

Le modele de donnees est compose de quatre entites principales lies par des relations definies via les attributs Doctrine ORM :

User 1 --- * BugReport (reporter)
 User 1 --- * BugReport (assignedDeveloper)
 User 1 --- * BugComment (author)
 Project 1 --- * BugReport
 BugReport 1 --- * BugComment

4.2 Description des entites

Entite User (app_user)

Champ	Type	Contraintes
id	int	Cle primaire, auto-generee
email	string(180)	Unique, NotBlank, Email
roles	json	Tableau de roles
password	string	Hache (auto hasher)
fullName	string(120)	NotBlank, max 120
isActive	boolean	Default: true
createdAt	DateTimeImmutable	Default: date courante

Entite BugReport

Champ	Type	Contraintes
id	int	Cle primaire
title	string(180)	NotBlank, max 180
description	text	NotBlank
stepsToReproduce	text?	Optionnel
expectedResult	text?	Optionnel
actualResult	text?	Optionnel
priority	enum(BugPriority)	Default: Medium
status	enum(BugStatus)	Default: Open
screenshotFilename	string(255)?	Optionnel
createdAt	DateTimeImmutable	Date de creation

updatedAt	DateTimeImmutable	Derniere mise a jour
openedAt	DateTimeImmutable?	Date d'ouverture
treatedAt	DateTimeImmutable?	Date de traitement
closedAt	DateTimeImmutable?	Date de fermeture
project_id	FK -> Project	NotNull
reporter_id	FK -> User	NotNull
assignedDeveloper_id	FK -> User	Optionnel

Entite BugComment

Champ	Type	Contraintes
id	int	Cle primaire
content	text	NotBlank
createdAt	DateTimeImmutable	Date de creation
bugReport_id	FK -> BugReport	NotNull, onDelete CASCADE
author_id	FK -> User	NotNull

Entite Project

Champ	Type	Contraintes
id	int	Cle primaire
name	string(120)	NotBlank, max 120
description	text?	Optionnel
platform	string(60)	NotBlank (Web, Mobile, API)
isActive	boolean	Default: true
createdAt	DateTimeImmutable	Date de creation
updatedAt	DateTimeImmutable	Derniere mise a jour

4.3 Enums et workflow des statuts

Les enumerations PHP 8.1+ assurent la type-safety des valeurs :

BugStatus (Statut du bug)

Valeur	Label	Description
open	Open	Bug nouveau, en attente de traitement
in_progress	In Progress	Bug en cours de traitement
fixed	Fixed	Bug corrige (supprime du systeme)
rejected	Rejected	Bug rejete (non reproductible, etc.)

closed	Closed	Bug ferme definitivement
--------	--------	--------------------------

Workflow des statuts : Open -> InProgress -> Fixed | Rejected -> Closed. Lorsqu'un bug passe a 'Fixed', il est automatiquement supprime du systeme avec sa capture d'ecran associee.

BugPriority (Priorite du bug)

Valeur	Label
low	Low (Basse)
medium	Medium (Moyenne)
high	High (Haute)
critical	Critical (Critique)

Chapitre 5 : Fonctionnalités détaillées

5.1 Authentification et gestion des sessions

L'authentification est implementee via le composant Security de Symfony. Les utilisateurs se connectent avec leur adresse email et mot de passe. Le systeme utilise un formulaire de login avec protection CSRF.

Les mots de passe sont haches automatiquement avec l'algorithme auto de Symfony (Argon2i/BCrypt selon le systeme). Un UserChecker verifie que le compte est actif avant chaque requete authentifiee.

Apres la connexion, l'utilisateur est redirige vers son tableau de bord personnalise selon son role : /admin/dashboard, /developer/dashboard, ou /client/dashboard.

5.2 Tableaux de bord par role

Tableau de bord Administrateur

Le tableau de bord administrateur affiche des metriques globales : nombre total de bugs, bugs ouverts, en cours, fixes, critiques et non assignes. Il presente egalement des tableaux de repartition par statut et par projet, ainsi que des liens rapides vers la gestion des bugs, des projets et des utilisateurs.

Tableau de bord Developpeur

Le developpeur voit le nombre de bugs qui lui sont assignes, avec des liens rapides vers la liste des bugs et la creation d'un nouveau bug.

Tableau de bord Client

Le client voit le nombre de bugs qu'il a signales, avec des liens rapides vers la liste de ses bugs et la creation d'un nouveau bug.

5.3 Gestion des rapports de bugs

La creation d'un rapport de bug inclut les champs suivants : titre, description detaillee, etapes pour reproduire le bug, resultat attendu, resultat obtenu, priorite, projet associe, et capture d'ecran optionnelle.

Chaque bug est automatiquement lie a l'utilisateur rapporteur et recoit le statut 'Open' a sa creation. Les timestamps sont tracks automatiquement : date de creation, date d'ouverture, date de traitement, et date de fermeture.

La vue detaillee d'un bug affiche toutes les informations, un chronometre de suivi, le formulaire de mise a jour du statut (pour les developpeurs), et la liste des commentaires.

5.4 Systeme de filtrage

La page d'index des bugs offre 7 filtres combinaisons :

- Recherche par mot-cle (titre et description)
- Filtrage par projet
- Filtrage par statut (Open, InProgress, Fixed, Rejected, Closed)
- Filtrage par priorite (Low, Medium, High, Critical)
- Filtrage par developpeur assigne (admin/developpeur uniquement)
- Filtrage par date de debut
- Filtrage par date de fin

Les clients ne voient que leurs propres bugs, tandis que les admins et developpeurs voient tous les bugs du systeme.

5.5 Gestion des projets et utilisateurs

L'administrateur peut creer, modifier et supprimer des projets. Chaque projet a un nom, une description, une plateforme (Web, Mobile, API) et un statut actif/inactif.

La gestion des utilisateurs permet a l'administrateur de creer de nouveaux comptes, modifier les informations et les roles, et desactiver des comptes. Les mots de passe sont haches lors de la creation et de la modification.

5.6 Systeme de commentaires

Chaque bug dispose d'un systeme de commentaires permettant aux differents participants de discuter du bug. Les commentaires sont affiches dans l'ordre chronologique et enregistrent l'auteur, le contenu et la date.

5.7 Upload de captures d'ecran

Les utilisateurs peuvent joindre une capture d'ecran lors de la creation d'un bug. Le systeme accepte les formats JPG, PNG et WEBP avec une taille maximale de 2 Mo.

Les fichiers sont stockes dans le repertoire `var/uploads/screenshots/` avec un nom de fichier genere aleatoirement (hash MD5) pour eviter les collisions. Les fichiers ne sont jamais accessibles directement depuis le web ; ils sont servis via une route protegee qui verifie les droits d'accès.

Chapitre 6 : Securite

6.1 Authentification et hachage des mots de passe

Les mots de passe sont stockes de maniere securisee grace au composant PasswordHasher de Symfony. L'algorithme auto utilise Argon2i ou BCrypt selon les capacites du systeme, avec un cout adaptatif pour empecher les attaques par force brute.

Aucun mot de passe n'est jamais stocke en clair. Le hachage est effectue lors de la creation et de la modification d'un utilisateur.

6.2 Controle d'accès basé sur les rôles

Le systeme de securite implemente une hierarchie de rôles :

- ROLE_ADMIN : acces complet, herite de ROLE_DEVELOPER, ROLE_CLIENT et ROLE_USER
- ROLE_DEVELOPER : gestion des bugs assignes, herite de ROLE_USER
- ROLE_CLIENT : creation et consultation de ses propres bugs
- ROLE_USER : acces de base a l'application

Les regles d'accès sont definies dans security.yaml et appliquees via les attributs #[IsGranted] sur les controleurs. Le UserChecker verifie egalement que le compte est actif avant chaque requete.

6.3 Protection CSRF

La protection CSRF est implementee via le composant CSRF de Symfony avec le mecanisme de double-submit cookie. Un controleur JavaScript Stimulus gere l'envoi des tokens CSRF lors des soumissions de formulaires via Hotwired Turbo.

Tous les formulaires de suppression (bugs, projets, utilisateurs) et le formulaire de login sont proteges par des tokens CSRF valides cote serveur.

6.4 Validation des entrees

Toutes les donnees utilisateur sont validees via les contraintes Doctrine et Symfony Validator : NotBlank, Email, Length, UniqueEntity, etc. Les types de formulaire Symfony assurent une validation supplementaire avant tout traitement metier.

Les parametres de requete sont filtres et casts avant utilisation. Les identifiants numeriques sont verifies avec ctype_digit(). Les dates sont parsees avec un format strict (Y-m-d) et validees.

Chapitre 7 : Tests

7.1 Tests unitaires

Les tests unitaires couvrent le comportement des entites metier, en particulier le mecanisme de timestamps automatiques sur BugReport. Le fichier BugReportTest.php teste :

- La mise a jour automatique de closedAt lors du changement de statut
- La reinitialisation de closedAt lors de la reouverture d'un bug

7.2 Tests fonctionnels

Les tests fonctionnels utilisent le composant BrowserKit de Symfony pour simuler des requetes HTTP. Le fichier BugReportControllerTest.php verifie :

- La redirection vers /login pour les utilisateurs non authentifies

La suite de tests peut etre etendue pour couvrir les scenarios suivants :

- Authentification reussie et echouee
- Creation d'un bug par un client
- Mise a jour du statut par un developpeur
- Gestion des projets par un administrateur
- Restriction d'accès selon les roles

La commande pour executer les tests : `php bin/phpunit`

Resultat actuel : 3 tests, 6 assertions, tous en succes.

Conclusion generale

Ce stage m'a permis de developper une application web complete en utilisant le framework Symfony 7.4. J'ai pu appliquer les concepts theoriques appris en formation : architecture MVC, programmation orientee objet, base de donnees relationnelles, securite web, et tests logiciels.

Les principaux apprentissages de ce stage sont :

- La maitrise du framework Symfony et son ecosysteme (Doctrine, Twig, Security)
- La conception et l'implementation d'un modele de donnees relationnel
- La gestion des droits d'accès base sur les roles (RBAC)
- La protection contre les vulnerabilites courantes (CSRF, injection, XSS)
- L'importance des tests pour garantir la fiabilite du code
- La gestion de projet et l'organisation du code source

Cette plateforme constitue une base solide qui peut etre etendue avec des fonctionnalites additionnelles telles que les notifications par email, l'export de rapports, ou une API REST pour l'integration avec d'autres outils.

Annexes

Annexe A : Comptes de test

Role	Email	Mot de passe
Admin	admin@example.com	admin
Developpeur	dev@example.com	password123
Client	client@example.com	password123

Annexe B : Projets de test

Nom	Plateforme
Company Website	Web
Mobile Ordering App	Mobile
Internal CRM	Web
Partner API	API

Annexe C : Routes principales

Route	Methode	Description
/	GET	Redirection basee sur le role
/login	GET POST	Page de connexion
/admin/dashboard	GET	Tableau de bord admin
/developer/dashboard	GET	Tableau de bord dev
/client/dashboard	GET	Tableau de bord client
/bugs	GET	Liste des bugs (filtres)
/bugs/new	GET POST	Creation d'un bug
/bugs/{id}	GET POST	Detail d'un bug
/bugs/{id}/manage	GET POST	Gestion admin d'un bug
/bugs/{id}/status	POST	Mise a jour du statut
/bugs/{id}/delete	POST	Suppression d'un bug
/admin/projects	GET	Liste des projets
/admin/users	GET	Liste des utilisateurs

Annexe D : Dependances principales

doctrine/doctrine-bundle ^3.2, doctrine/doctrine-migrations-bundle ^4.0, doctrine/orm ^3.6, symfony/asset 7.4.*, symfony/asset-mapper 7.4.*, symfony/console 7.4.*, symfony/dotenv 7.4.*, symfony/flex ^2, symfony/form 7.4.*, symfony/framework-bundle 7.4.*, symfony/monolog-bundle ^3.0|^4.0, symfony/property-access 7.4.*, symfony/property-info 7.4.*, symfony/runtime 7.4.*, symfony/security-bundle 7.4.*, symfony/stimulus-bundle ^3.2, symfony/string 7.4.*, symfony/translation 7.4.*, symfony/twig-bundle 7.4.*, symfony/ux-turbo ^3.2, symfony/validator 7.4.*, symfony/yaml 7.4.*, twig/extra-bundle ^2.12|^3.0, twig/twig ^2.12|^3.0